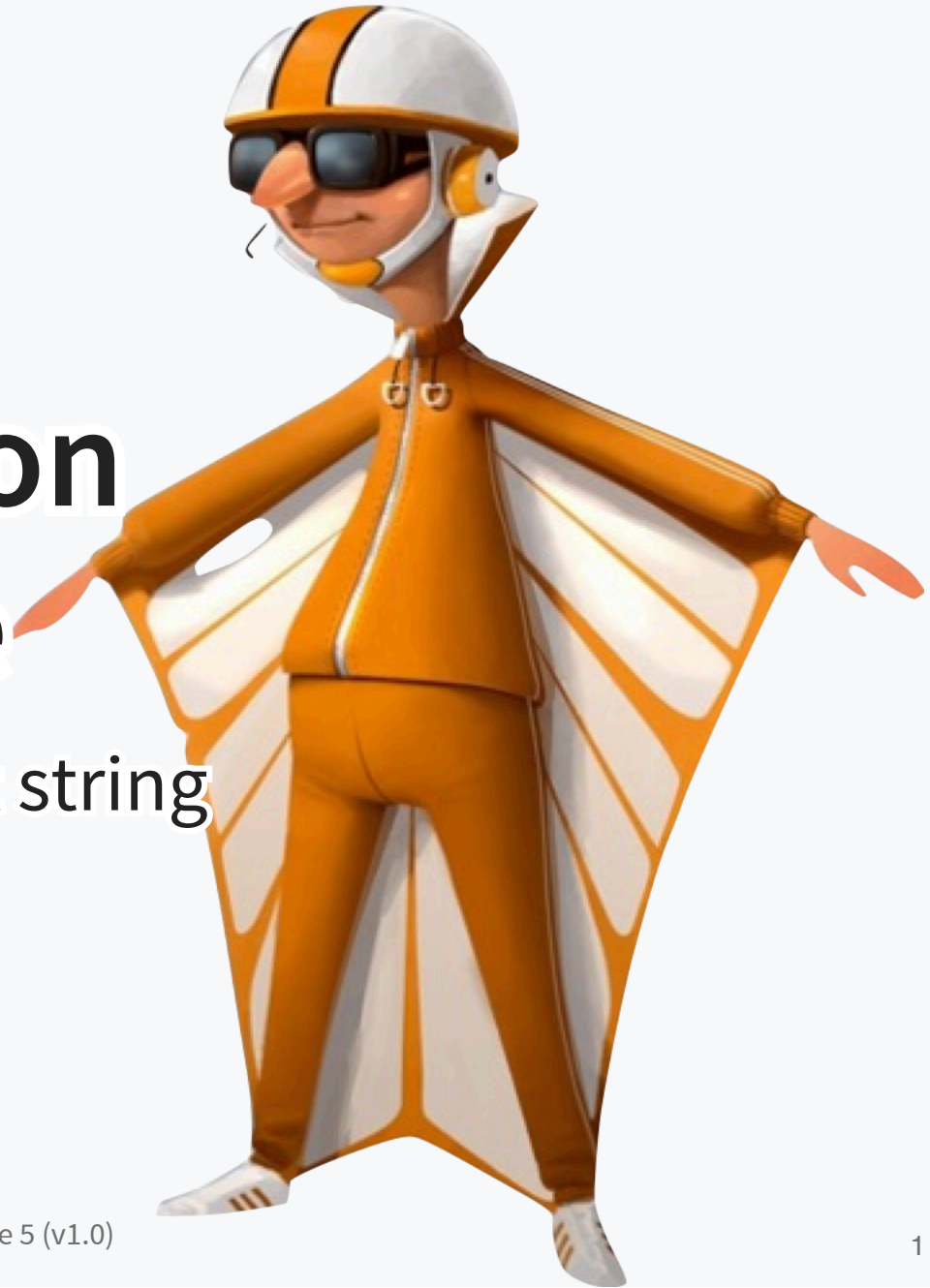
Programmation procédurale

Chapitre 5 : Arrays, vector et string

Automne 2026



Structure du chapitre

- Tableaux en C++, conteneurs

1. `std::array<T,n>`
2. `std::vector<T>`
3. Tableaux multi-dimensionnels
4. Le type `char` et `<cctype>`
5. Chaînes littérales
6. La classe `std::string`
7. Opérateurs et méthodes de `std::string`



Illustrations des sections extraites de https://www.ikea.com

Tableaux



- Pour stocker **plusieurs** éléments de **même type**, la méthode la plus simple consiste à les stocker dans un **tableau** (English : array).
- On accède aux **éléments** du tableau via un **indice**, qui indique leur **position** dans le tableau :

```
t[0] = 3;         // écrit 3 à la première position du tableau
cout << t[2];     // affiche l'élément à la troisième position
```

- Le **premier** élément est à l'**indice 0** dans la plupart des langages.
- Les **éléments** du tableau sont placés **consécutivement en mémoire**. Par exemple en C++, pour un tableau `t` d'éléments de type `int` :

```
for (int i = 0; i < 4; ++i) {
   cout << &(t[i]) << endl;
}
cout << "sizeof(int) = " << sizeof(int);
```

```
0x142604410
0x142604414
0x142604418
0x14260441c
sizeof(int) = 4
```

+4
+4
+4

Tableaux en C++



- Le C++ propose deux méthodes pour créer un tableau de `n` entiers :

```
array<int, n> a;   // taille n (fixe = constante), connue à la compilation

vector<int> v(n);  // taille n (variable), connue à l'exécution
```

- `std::array` est utilisé pour des tableaux dont la **taille** (nombre d'éléments) est **fixée à la compilation**.
- `std::vector` est utilisé pour des tableaux dont la taille n'est **connue qu'à l'exécution** ou dont la taille **peut varier**.

Note : conteneurs en C++

- La bibliothèque standard propose des **alternatives** aux tableaux pour stocker des données multiples : les **conteneurs**.
- Ils stockent leurs éléments dans des **structures de données** plus complexes : tableau de tableaux, listes chaînées, arbres binaires de recherche, tables de hachage, …
- Ils offrent d'autres **compromis** entre coût en mémoire, vitesse d'exécution, fonctionnalités, …
- Leur étude détaillée se fera lors du cours d'**Algorithmes et Structures de Données**.

Container class templates

Sequence containers:

array C++11	Array class (class template)
vector	Vector (class template)
deque	Double ended queue (class template)
forward_list C++11	Forward list (class template)
list	List (class template)

Container adaptors:

stack	LIFO stack (class template)
queue	FIFO queue (class template)
priority_queue	Priority queue (class template)

Associative containers:

set	Set (class template)
multiset	Multiple-key set (class template)
map	Map (class template)
multimap	Multiple-key map (class template)

Unordered associative containers:

unordered_set C++11	Unordered Set (class template)
unordered_multiset C++11	Unordered Multiset (class template)
unordered_map C++11	Unordered Map (class template)
unordered_multimap C++11	Unordered Multimap (class template)



1. std::array<T,n>



Déclaration



- La classe générique `std::array` est définie dans le header `<array>`.
- Le type `T` et le nombre `n` d'éléments font partie du type déclaré, selon la syntaxe :

```
std::array<T,n> nom_de_variable;
```

- Exemples :

```
std::array<int, 10> a1;         // tableau de 10 int
std::array<int, 42> a2;         // tableau de 42 int
std::array<double, 5> a3;       // tableau de 5 double
std::array<std::string, 7> a4; // tableau de 7 std::string
// tableau de 3 tableaux de 4 double
std::array<std::array<double, 4>, 3> a5;
```

Déclaration sans initialisation



- Sans initialisation explicite, les éléments d'un `std::array<T,n>` déclaré en variable locale ont une valeur indéterminée si le type `T` ne s'initialise pas par défaut.

```
// variable locale : 10 entiers de valeurs indéterminées
std::array<int, 10> a1;
// variable locale : 7 std::string initialisées vides
std::array<std::string, 7> a2;
```

- Le comportement est identique à celui de la déclaration d'un élément seul :

```
int e1;            // variable locale : 1 entier de valeur indéterminée
std::string e2;    // variable locale : 1 std::string initialisée vide
```

- Comme pour les variables simples, un `std::array<T,n>` alloué **statiquement** (variable locale **statique** ou **globale**) a ses éléments **initialisés par défaut** (à zéro pour les types numériques).

HEIG VD



Initialisation par agrégat

- On peut donner une valeur initiale aux éléments en utilisant un **agrégat** entre accolades contenant ces valeurs initiales :

```
array<int, 4> a1 {3, 1, 4, 1};     // le tableau contient [3, 1, 4, 1]
```

- L'agrégat **peut contenir moins d'éléments** que le `std::array<T,n>`, les premiers éléments sont initialisés par l'agrégat, les suivants par défaut :

```
array<int, 4> a3 {3, 1};           // le tableau contient [3, 1, 0, 0]
array<int, 4> a4 {};               // le tableau contient [0, 0, 0, 0]
```

- L'agrégat **ne peut pas contenir plus d'éléments** que le `std::array<T,n>` :

```
array<int, 4> a5 {3, 1, 4, 1, 5};  // error: excess elements in struct initializer
```

Initialisation par copie / affectation



- Enfin, on peut **initialiser** un `std::array<T,n>` en **copiant** le contenu d'un autre `std::array<T,n>` **du même type**, i.e. avec les mêmes type `T` et nombre d'éléments `n`.

```
array<int, 4> a1 {3, 1, 4, 5};

array<int, 4> a2 = a1;        // a2 contient [3, 1, 4, 5]

array<int, 5> a3 = a1;
  // error: no viable conversion from 'array<[...], 4>'
  // to 'array<[...], 5>'

array<unsigned, 4> a4 = a1;
  // error: no viable conversion from 'array<int, [...]>'
  // to 'array<unsigned int, [...]>'
```

- Notons que l'on peut aussi copier dans un `std::array<T,n>` existant par **affectation** :

```
array<int, 4> a5;             // a5 indéterminé (variable locale)
a5 = a1;                      // a5 contient [3, 1, 4, 5]
```

HEIG VD



Remplissage

- L'initialisation par agrégat vide permet de remplir un `std::array<T,n>` de valeurs par défaut, i.e. `T()`.
  - `T()` est l'appel au **constructeur par défaut** du type `T` (chapitre 8) : 0 pour les types numériques, la chaîne vide pour `std::string`
- La méthode `.fill(val)` permet de remplir un `std::array<T,n>` existant avec la même valeur `val` pour ses `n` éléments.

```
array<int, 4> a;        // 4 entiers indéterminés (variable locale)
a.fill(42);             // a contient [42, 42, 42, 42]
a = {};                 // a contient [0, 0, 0, 0]
a.fill(-1);             // a contient [-1, -1, -1, -1]
a.fill(int());          // int() vaut 0 : a contient [0, 0, 0, 0]
```



Accès aux éléments d'un `std::array<T,n>`

- Lire ou écrire un élément se fait via son **indice** `i`, de type `size_t` (non signé) :
  - Avec l'opérateur `[i]`, qui ne vérifie pas que l'indice est correct
  - Avec la méthode `.at(i)`, qui lève une exception si `i >= n` :
    - Notons qu'un indice `i` négatif est `>= n` une fois converti en `size_t`
- Les premier et dernier éléments peuvent être accédés via les méthodes `.front()` et `.back()`.
- `a[i]` et `a.at(i)` désignent l'élément lui-même, pas une copie : on peut donc écrire `a[i] = 42`.

```
array<int, 3> a {3, 1, 4};
cout << a[0] << a.at(0) << a.front();    // affiche 333
cout << a[1] << a.at(1);                 // affiche 11
cout << a[2] << a.at(2) << a.back();     // affiche 444
cout << a[3];    // comportement indéterminé
cout << a.at(3); // libc++abi: terminating due to uncaught exception
                 // of type std::out_of_range: array::at
```

Parcours par indice croissant



- La méthode `.size()` retourne le nombre d'éléments, de type `size_t`.
  - rappel (chapitre 2) : `size_t` est un entier **non signé**, assez grand pour une taille de tableau
- Une boucle sur tous les éléments par indice croissant s'écrit donc :

```
array<int, 4> a {3, 1, 4, 1};

for (size_t i = 0; i < a.size(); ++i)
   cout << a[i] << ' ';                    // affiche 3 1 4 1

for (size_t i = 0; i < a.size(); ++i)
   a[i] *= a[i];                           // a contient [9, 1, 16, 1]
for (size_t i = 0; i < a.size(); ++i)
   cout << a.at(i) << ' ';                 // affiche 9 1 16 1
```



Parcours par indice décroissant

- Le **premier élément à traiter** est à l'indice `i = a.size()-1` et le **dernier à traiter** est à `i = 0`.
- Le **premier élément à ne pas traiter** `(i = -1)` … dépend du type choisi pour l'indice :
  - Avec un type `unsigned`, la **condition de continuation** de la boucle reste `i < a.size()`. Une condition `i >= 0` serait toujours vraie et entraînerait une boucle infinie.

```
array<int, 4> a {3, 1, 4, 1};
// rappel : i est non signé, quand i passe sous 0 il vaut 2^64 - 1 (modulo 2^n)
for (size_t i = a.size()-1; i < a.size(); --i)
   cout << a[i] << ' ';                          // affiche 1 4 1 3
for (size_t i = a.size()-1; i >= 0; --i)
   cout << a[i] << ' ';              // boucle infinie, comportement indéterminé
```

- Avec un type `signed` (ici `ptrdiff_t`, l'entier signé de même taille que `size_t`), la **condition de continuation** de la boucle **`for`** peut être soit `i >= 0` (plus intuitif), soit `i < a.size()` (par conversion implicite de `i` en `size_t`).

```
// conversion implicite de i (ptrdiff_t) en size_t (non signé)
for (ptrdiff_t i = a.size()-1; i < a.size(); --i)
   cout << a[i] << ' ';                          // affiche 1 4 1 3
for (ptrdiff_t i = a.size()-1; i >= 0; --i)   // comparaison signée
   cout << a[i] << ' ';                          // affiche 1 4 1 3
```

Pour aller plus loin : indices signés ou non signés ?



- La **Standard Template Library** d'Alexander Stepanov utilise le type non signé `size_t` pour les indices des conteneurs.
  - Avantages : une seule comparaison nécessaire pour s'assurer qu'un indice est dans l'intervalle `[0,n[`. Pas d'avertissement pour conversion implicite `signed` → `unsigned` dans des expressions telles que `i < a.size()` ou `a[i]`.
  - Inconvénient : essayez d'écrire un parcours de l'intervalle d'indices `[a,b]` par indice décroissant avec `a` qui peut être nul ou pas …



- Les **C++ Core Guidelines** de Bjarne Stroustrup conseillent d'utiliser le type signé `gsl::index` défini par la **Guidelines Support Library** comme :

```
using index = std::ptrdiff_t;
```

¯_(ツ)_/¯

ES.107: Don't use unsigned for subscripts, prefer `gsl::index`

- Pas de consensus : dans ce cours, on indexe avec `size_t`, en faisant attention aux parcours décroissants.

Parcours complet via boucle `for( : )`



- Pour parcourir **tous les éléments** d'un conteneur par indice croissant mais **sans connaître l'indice associé** à chaque élément.
- La méthode la plus **propre** et la plus **sûre** consiste à utiliser `for( : )` :

```
array<double, 4> a {3., 1., 4., 2.};

for (double e : a)
   cout << e;       // affiche 3142
```

- Attention, ci-dessus e est une **copie** des éléments de a. Pour les modifier, il faut plutôt en prendre une **référence**.

```
for (double e : a)
   e *= e;          // a est inchangé et contient [3., 1., 4., 2.]

for (double& e : a)
   e *= e;          // a contient [9., 1., 16., 4.]
```

for(:)



- Quand les éléments du conteneur sont eux-mêmes de **type composé**, il faut plutôt choisir entre **référence** et **référence constante** pour éviter les copies inutiles.

```
array<string, 4> a {"Hello", ", ", "world", "!\n"};

for (const string& s : a)
   cout << s;                        // affiche "Hello, world!\n"
                                     // sans copie inutile de string
for (string& s : a)
   if (s == "world")
      s = "class of PRG";            // modifie a[2]

for (const string& s : a)
   cout << s;                        // affiche "Hello, class of PRG!\n"
```



Comparaisons

- Tous les opérateurs de comparaison existent entre deux `std::array<T,n>` de même type : `==`, `!=`, `<`, `>`, `<=`, `>=`.
- Deux `std::array<T,n>` sont égaux si tous les éléments de même indice le sont deux à deux.
- Les comparaisons d'ordre `<`, `>`, `<=`, `>=` suivent l'**ordre lexicographique** : on compare élément par élément, le premier qui diffère décide, comme pour les `std::string`.

```
array<int,4> a {1, 2, 3, 4};
array<int,4> b {1, 2, 3, 4};
array<int,4> c {1, 2, 4, 3};

bool vrai = (a == b) and (a != c) and (a < c) and (c > a);

bool faux = (a != b) or (a == c) or (a >= c) or (c <= a);
```

Passage en paramètre / en retour



- Un `std::array<T,n>` se comporte comme les autres types du C++.
- On peut le passer par **référence**, par **référence constante**, voire par **valeur** si une copie est nécessaire.

```
// total des éléments
long long somme(const array<int,5>& a) {
   long long s = 0;
   for (int e : a) s += e;
   return s;
}

// remplace chaque élément par la somme des précéder
void somme_cumulee_1(array<int,5>& a) {
   for (size_t i = 1; i < a.size(); ++i)
      a[i] += a[i-1];
}
```

```
array<int, 5> a {1, 2, 3, 4, 5};

long long s = somme(a);
// s = 15, a inchangé

somme_cumulee_1(a);
// a = {1, 3, 6, 10, 15}
```

Passage en paramètre / en retour



- On peut aussi le retourner avec **`return`**.

```
array<int,5> somme_cumulee_2(
                    const array<int,5>& a) {
   array<int,5> b;
   b.front() = a.front();
   for (size_t i = 1; i < b.size(); ++i)
      b[i] = b[i-1] + a[i];
   return b;
}

array<int,5> somme_cumulee_3(array<int,5> a) {
   for (size_t i = 1; i < a.size(); ++i)
      a[i] += a[i-1];
   return a;
}
```

```
// a = {1, 3, 6, 10, 15}

array<int,5> b = somme_cumulee_2(a);
// b = {1, 4, 10, 20, 35}

array<int,5> c = somme_cumulee_3(a);
// c = {1, 4, 10, 20, 35}
// a inchangé
```

Passage en paramètre / en retour



- `std::array<T1,n1>` et `std::array<T2,n2>` étant des **types différents** dès que `T1 != T2` ou `n1 != n2`, il n'est pas possible à ce stade d'écrire des fonctions qui traitent des `std::array<T,n>` avec `T` et `n` quelconques.

2. `std::vector<T>`



`vector<T>` vs `array<T,n>`



- `std::array<T,n>` souffre de limitations importantes :
  - `n` doit être **constant**
  - `n` doit être calculable à la **compilation**
  - `std::array<T,n1>` et `std::array<T,n2>` sont deux **types différents**
- `std::vector<T>` permet de dépasser ces limitations :
  - en **allouant dynamiquement** le bloc de mémoire où sont stockés les éléments du tableau
  - en le **réallouant** si nécessaire de manière transparente pour l'utilisateur
  - au prix d'un léger **surcoût en mémoire**, typiquement de `3*sizeof(T*)`, comme expliqué au cours d'ASD.
- `std::vector<T>` offre :
  - la **même interface** que `std::array<T,n>` (sauf la méthode `fill`)
  - des méthodes supplémentaires permettant de **redimensionner** le tableau et d'y **ajouter** ou en **supprimer** des éléments.

Déclaration



- La classe générique `std::vector` est définie dans le header `<vector>`.
- Le type `T` des éléments fait partie du type déclaré, selon la syntaxe :

```
std::vector<T> nom_de_variable;
```

- Exemples :

```
std::vector<int> v1;           // tableau redimensionnable de 0 int
std::vector<double> v2;        // tableau redimensionnable de 0 double
std::vector<std::string> v3;   // tableau redimensionnable de 0 std::string
// tableau redimensionnable de 0 tableaux de 3 double
std::vector<std::array<double, 3>> v4;
```

- Ainsi initialisé par défaut, un `std::vector<T>` ne contient aucun élément :

```
v.size() == 0 and v.empty() == true
```

Initialisation - agrégat et copie



- Comme pour `std::array<T,n>`, les initialisations par agrégat et par copie sont disponibles :

```
vector<int> v1 {3, 1, 4, 2};          // v1 contient [3, 1, 4, 2]
vector<int> v2 = v1;                  // v2 contient [3, 1, 4, 2]
```

- Contrairement à `std::array<T,n>`, l'initialisation avec un agrégat partiel n'est pas possible, la taille du `std::vector<T>` étant déterminée par la taille de l'agrégat.

Initialisation par remplissage



- Il est également possible d'initialiser un `std::vector<T>` avec une taille et une valeur commune pour tous les éléments.
- Syntaxe : `std::vector<T> nom(taille, valeur);`
- Si elle n'est pas spécifiée, `valeur = T();`
- Exemples :

```
vector<int> v1(4, 10);   // v1 contient [10, 10, 10, 10]
vector<double> v3(3);    // v3 contient [0., 0., 0.]
vector<bool> v4(2);      // v4 contient [false, false]
```

`resize(…)` / `clear()`



- Les méthodes `resize` et `clear` modifient explicitement la taille du `vector`.
- Syntaxe :

```
void resize(size_t taille, const T& val = T());
void clear();                  // équivalent à resize(0);
```

- Quand la `taille` demandée est plus petite que précédemment, les éléments finaux sont supprimés.
- Quand elle est plus grande, les éléments ajoutés en fin prennent la valeur `val` si elle est spécifiée, ou celle par défaut `T()` sinon.
- Exemples :

```
vector<int> v {2, 3, 5};     // {2, 3, 5}
v.resize(5, 1);              // {2, 3, 5, 1, 1}
v.resize(2);                 // {2, 3}
v.resize(4);                 // {2, 3, 0, 0}
v.clear();                   // { }
```

`push_back(…)` / `pop_back()`



- Les méthodes `push_back` et `pop_back` modifient implicitement la taille du `vector` en ajoutant / supprimant un élément en position finale.
- Syntaxe :

```
void push_back(const T& val);
void pop_back();
```

- Exemples :

```
vector<int> v {2, 3, 5};   // {2, 3, 5}
v.push_back(1);            // {2, 3, 5, 1}
v.push_back(4);            // {2, 3, 5, 1, 4}
v.pop_back();              // {2, 3, 5, 1}
```

- Équivalence avec `resize` :

```
v.push_back(val);
v.pop_back();
```

```
v.resize(v.size()+1, val);
v.resize(v.size()-1);
```

Insertion



Pour insérer un élément à une position donnée, il faut :

- Incrémenter la taille du vecteur.
- Déplacer tous les éléments de cette position à la fin du vecteur d'une position vers l'arrière.
- Copier le nouvel élément à la place demandée :

```
void insertion(vector<int>& v, size_t pos, int val) {
   if (pos > v.size())    // position d'insertion illicite
      return;

   v.resize(v.size()+1);      // maintenant, v.size() >= 1
   for (size_t i = v.size()-1; i > pos; --i)
      v[i] = v[i-1];

   v[pos] = val;
}
```

- C'est une opération chère si `pos` est loin de la fin du vecteur.

Suppression

Pour supprimer un élément à une position donnée, il faut :

- Déplacer tous les éléments qui suivent cette position d'une position vers le début.
- Décrémenter la taille du vecteur :

```
void suppression(vector<int>& v, size_t pos) {

   if (pos >= v.size())  // position de suppression illicite
      return;

   for (size_t i = pos + 1; i < v.size(); ++i)
      v[i-1] = v[i];

   v.pop_back();
}
```

- Noter que les positions d'insertion et de suppression licites ne sont pas les mêmes.
- C'est une opération chère si `pos` est loin de la fin du vecteur.

`insert`, `erase`, `emplace`, `emplace_back`, ...



- Ces variantes reposent sur la notion d'itérateur, non couverte dans ce cours. Détails sur cppreference.com.

capacity()



- La **réallocation** de mémoire lors d'un changement de taille est une opération **coûteuse**. Il faut réserver une nouvelle zone mémoire capable de stocker tout le vecteur et y déplacer tous les éléments du vecteur.
- Pour être **efficace**, `std::vector<T>` ne **réalloue** jamais quand la taille diminue et **rarement** quand elle augmente. Pour cela, il distingue :

- la **taille** : nombre d'éléments stockés.
- la **capacité** : nombre d'éléments stockables dans la mémoire actuellement allouée.
- La plupart des `push_back` ont lieu avec taille < capacité, et consistent simplement à écrire dans le premier emplacement libre - à l'indice `v.size()` - puis à incrémenter la taille.

```
vector<int> v;
size_t c = v.capacity();
cout << c << ' ';
for (int i = 0; i < 1000; ++i) {
   v.push_back(i);
   if (c != v.capacity())
      cout << (c = v.capacity()) << ' ';
}
```

```
 0 1 2 4 8 16 32 64 128 256 512 1024
```

`reserve()`, `shrink_to_fit()`



- Les méthodes `reserve` et `shrink_to_fit` permettent de gérer manuellement la capacité. Elles n'ont **aucun effet sur le contenu** du `vector`, mais peuvent en **améliorer l'efficacité** en **temps** et en **mémoire** si elles sont bien utilisées.

- `void reserve(size_t c);` demande que la capacité soit d'au moins `c`. Elle n'a aucun effet si c'est déjà le cas. Pertinent si l'on sait à priori combien de `push_back` seront effectués.
- `void shrink_to_fit();` demande que la capacité soit diminuée pour être égale à la taille actuelle. La bibliothèque n'est pas obligée de le faire.

```cpp
vector<int> v;
v.reserve(1000);
size_t c = v.capacity();
cout << c << ' ';
for (int i = 0; i < 1000; ++i) {
   v.push_back(i);
   if (c != v.capacity())
      cout << (c = v.capacity()) << ' ';
}
```

```
1000
```

3. Tableaux multi-dimensionnels



Tableaux multi-dimensionnels



- Un tableau peut contenir des **éléments qui sont eux aussi des tableaux**. On parle alors de tableau multi-dimensionnel.
- Par exemple, voici les deux formes usuelles de déclarer et initialiser à 0 un tableau de **4 lignes** et **3 colonnes** de `double` :

```
array<array<double, 3>, 4> m1 {};

vector<vector<double>> m2(4, vector<double>(3, 0.));
```

- Pour de petits tableaux de taille fixe, `std::array` est préférable.
- Si le type des tableaux imbriqués est `std::vector`, le tableau multi-dimensionnel n'est pas nécessairement rectangulaire, chaque ligne pouvant avoir un nombre de colonnes différent.

Accès aux éléments

- On accède aux éléments avec autant d'opérateurs `[]` (ou de méthodes `.at()`) qu'il y a de dimensions. Par exemple :

```
array<array<double, 3>, 4> m {};
m[2][1] = 1;
```

- La signification des 2 indices est plus claire en observant que l'on peut aussi écrire la même opération en 2 lignes :

```
array<double, 3>& ligne = m[2];
ligne[1] = 1;
```

- Pour afficher tout le tableau 2d, on écrirait :

```
for (size_t i_ligne = 0; i_ligne < m.size(); ++i_ligne) {
   for (size_t i_colonne = 0; i_colonne < m[i_ligne].size(); ++i_colonne)
      cout << m[i_ligne][i_colonne] << ' ';
   cout << endl;
}
```

```
0 0 0
0 0 0
0 1 0
0 0 0
```

Utilisation d'alias de type

- Le code peut éventuellement être rendu plus compréhensible en utilisant les alias de type avec **`typedef`** (écriture héritée du C) ou **`using`** :

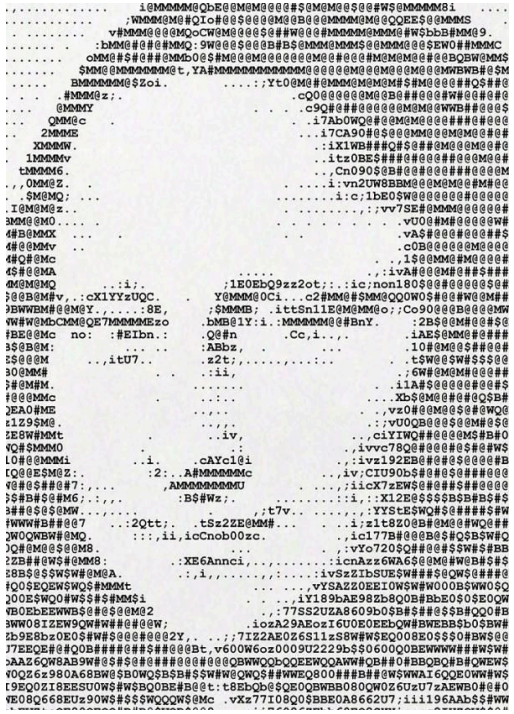
```
using Ligne3 = array<double, 3>;
using Matrice3x3 = array<Ligne3, 3>;

Matrice3x3 matrice{};   // tout à zéro

for (Ligne3& ligne : matrice) {
   for (double element : ligne)
      cout << element << ' ';
   cout << endl;
}
```

```
0 0 0
0 0 0
0 0 0
```

4. Le type char



Le type `char`



- Le type `char` permet de stocker des **entiers** stockés sur **1 Byte** et peut être précédé d'un **modificateur** ou non :
  - `signed char` pour des entiers entre **-128 et 127**
  - `unsigned char` pour des entiers entre **0 et 255**
  - `char` soit signé soit non signé selon le compilateur
- Il permet surtout de stocker des **caractères** via un **codage** qui fait correspondre caractères et valeurs numériques : **ASCII**.
- Le modificateur de signe **n'affecte pas le caractère** codé par les 8 bits du `char`.
- Les **constantes littérales** de type `char` s'écrivent entourées **d'apostrophes** :
  - `'a'` pour le a minuscule
  - `'3'` pour le caractère 3 (différent de la valeur entière 3)

Code ASCII

Codes de contrôle

Chiffres

Lettres majuscules

Lettres minuscules

Caractères spéciaux

ISO-8859-1																
	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8x	PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTS	PLD	PLU	RI	SS2	SS3
9x	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SGCI	SCI	CSI	ST	OSC	PM	APC
Ax	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
Bx	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
Cx	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Dx	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
Ex	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
Fx	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Affichage

- Le type transmis au flux par l'opérateur de flux (<<) détermine si on affiche le caractère ou la valeur numérique correspondante.
- Attention aux promotions de type !

```
cout << 'A' << " "
     << static_cast<int>('A') << endl;

cout << static_cast<char>(0x41) << " "
     << 0x41 << endl;

cout << 'D' + 'a' - 'A' << " "
     << char('D' + 'a' - 'A') << endl;
```

```
A 65
A 65
100 d
```



Quelques constantes particulières

Constante	Signification
`\n`	passage à la ligne
`\r`	retour chariot « CR »
`\t`	tabulation horizontale
`\v`	tabulation verticale
`\a`	alerte sonore
`\\`	barre oblique inverse « \ »
`\'`	caractère apostrophe « ’ »
`\"`	caractère guillemet « ” »
`\b`	retour arrière « backspace »
`\f`	saut de page « form feed »

Caractères - catégories



- La librairie <cctype> met à disposition des **fonctions** qui permettent de savoir à quelle **catégorie** appartient un caractère :

```
int isxxx(int c);
```

- Ces fonctions retournent un entier (pour des raisons historiques) :
  - non nul (**true**) si le caractère c est **xxx**
  - et zéro (**false**) sinon

http://www.cplusplus.com/reference/cctype

Caractères - catégories

`int isalnum(int c);`	est une lettre de l'alphabet ou un chiffre
`int isalpha(int c);`	est une lettre de l'alphabet (minuscule ou majuscule)
`int iscntrl(int c);`	est un caractère de contrôle (code 0 à 31 et 127 (DEL))
`int isdigit(int c);`	est un chiffre
`int isxdigit(int c);`	est un chiffre hexadécimal
`int isgraph(int c);`	est affichable et non blanc (code 33 à 126)

Caractères - catégories



- `<cctype>` fournit également deux fonctions de **transformation** :

```
int tolower(int c);
int toupper(int c);
```

qui retournent respectivement la **minuscule** et la **majuscule** du caractère transmis.

- Si la transformation n'est **pas possible** (caractères non alphabétiques), elles retournent le **caractère original inchangé**.

5. Chaînes littérales



Quand il y a plus d'un caractère…

- Les chaînes de caractères littérales s'écrivent entourées de **doubles guillemets**.
- Pour écrire le caractère " au milieu d'une chaîne, on le précède d'un backslash. Pour écrire un backslash, on le précède également d'un backslash.
- On peut écrire une **chaîne sur plusieurs lignes** en finissant chaque ligne par \.
- Il est possible d'écrire une chaîne en plusieurs parties qui se suivent.

```
int main() {
   cout << "Hello, World !" << endl;

   cout << " \" et \\ au milieu d'une chaine"
        << endl;

   cout << "Une chaine \
ecrite sur plusieurs \
lignes" << endl;

   cout << "Une"
           " chaine "
           "en " "plusieurs"
           "parties" << endl;
}
```

```
Hello, World !
 " et \ au milieu d'une chaine
Une chaine ecrite sur plusieurs lignes
Une chaine en plusieursparties
```



Note

- Notons que ces constantes littérales sont de type `const char*` et non du type `string`. Ce sont des tableaux de `char`, terminés conventionnellement par le caractère zéro. Le traitement de ce type de chaînes sera étudié dans le cours PAY.
- En pratique, on peut presque toujours ignorer cela, vu qu'il y aura **conversion implicite** vers le type `string` quand c'est nécessaire.
- Il est également possible d'effectuer une **conversion explicite** vers le type `string` :

```
string s1 = string("hello");
```

6. Classe `std::string`





La classe `std::string`

- On peut stocker une chaîne de caractères dans une variable de type `std::string`.
- Ce n'est pas un type fondamental du C++. Il est défini dans la librairie `<string>`, qu'il faut inclure.

```
#include <string>
using namespace std;

…

   string s = "Hello, World!";
```

http://www.cplusplus.com/reference/string

Initialisation



- Contrairement aux types simples, une variable non initialisée **est définie** :

```
string s0; // contient une chaine vide
```

- On peut initialiser les variables de type `string` des trois manières habituelles, par **affectation**, entre **parenthèses** ou entre **accolades** :
  - avec une **chaîne littérale** en paramètre :

```
string s1 = "Hello, World!";
string s2("Hello, World!");
string s3{"Hello, World!"};
```

- ou avec un autre `std::string` en paramètre qui est **copié** :

```
string s4 = s1;
string s5(s1);
string s6{s1};
```

Constructeurs



- Il est également possible d'initialiser un `std::string` **avec plusieurs paramètres**. On utilise alors la syntaxe de constructeur (avec parenthèses).
- Le constructeur de remplissage `string(size_t n, char c)` crée une chaîne contenant `n` fois le caractère `c` :

```
string s1(6, 'a'); // "aaaaaa"
```



Constructeurs par sous-chaîne

- Le constructeur `string(string str, size_t pos, size_t len)` crée une chaîne de `len` caractères copiés à partir du caractère en position `pos` de la chaîne `str` :
  - si `len` n'est pas précisé, `str` est copiée jusqu'à sa fin
  - on numérote les positions **à partir de 0** :

```
// indices    0123456789*12
string hello("Hello, World!");

string s1(hello, 0, 4);        // contient la chaîne "Hell"
string s2(hello, 7, 5);        // contient la chaîne "World"
string s3(hello, 7);           // contient la chaîne "World!"
string s4(hello);              // contient la chaîne "Hello, World!"
```

7. Opérateurs et méthodes



René Descartes

Affectation - opérateur =



- **L'opérateur =** permet d'affecter une nouvelle valeur à une `string`.
- Il **convertit implicitement** des expressions de type `char` ou des chaînes comme en C, y compris les constantes littérales.

```
string str1, str2, str3;
str1 = "Test string: "; // chaîne C littérale
str2 = 'x';             // caractère
str3 = str1;            // string
```

Concaténation - l'opérateur +



- L'opérateur + permet de concaténer deux chaînes :
  - Un des 2 paramètres peut être une chaîne littérale
  - Ou un caractère seul
- Par contre, on ne peut pas l'utiliser ✘ pour concaténer :
  - deux chaînes littérales
  - un `std::string` et un entier
  - une chaîne littérale et un `char` : là, + compile, mais fait de l'arithmétique d'adresse et le résultat est faux

```
string hello("Hello, ");
string world("World!");
string s1 = hello + world;     // "Hello, World!"
```

```
string s2 = "Hello, " + world; // "Hello, World!"
string s3 = hello + "World!";  // "Hello, World!"
```

```
string s4 = hello + 'W';       // "Hello, W"
string s5 = 'W' + hello;       // "WHello, "
```

✘

```
string erreur1 = "Hello, " + "World!";
  // ne compile pas
string erreur2 = hello + 5;
  // ne compile pas
string erreur3 = "Hello, " + 'W';
  // comportement indéfini
```

Concaténation - l'opérateur +=



- Comme pour les opérateurs sur les entiers et les réels, il y a un **opérateur auto-affecté** correspondant, qui accepte les `char`, les `string` et les chaînes littérales.

```
string str("Hello");

str += ',';          // même effet que str = str + ',';
                     // str contient "Hello,"

str += " World";     // même effet que str = str + " World";
                     // str contient maintenant "Hello, World"

string exclamation("!");
str += exclamation; // même effet que str = str + exclamation;
                     // str contient maintenant "Hello, World!"
```

Accès aux caractères - opérateur [ ]



- Pour une chaîne `str` et un entier `i`, l'expression `str[i]` permet d'accéder en lecture comme en écriture au `i`ème caractère - en numérotant depuis 0.

```
string hello("Hello, World!");
char fifth = hello[4];
hello[4] = ' ';

cout << hello << endl;
cout << fifth << " remplace par un blanc" << endl;
```

```
Hell , World!
o remplace par un blanc
```

Nombre de caractères



- On peut demander la **longueur** d'une chaîne avec les méthodes `.length()` ou `.size()`.
- La valeur retournée est de type `size_t`, un type entier non signé dont C++ garantit qu'il est assez grand pour stocker la taille de toute chaîne, tableau, objet, …

```cpp
string s("Hello");
size_t longueur = s.length(); // 5
size_t taille   = s.size();   // 5 aussi
```

- La méthode `.empty()` indique une longueur nulle.

```cpp
string s("Hello");

bool vide1 = s.size() == 0;  // false
bool vide2 = s.empty();      // idem, mais plus clair
```

Sous-chaîne



- La méthode `.substr(size_t pos, size_t len);` permet d'extraire une sous-chaîne de `len` caractères en commençant à la position `pos` :
  - si `len` manque, on extrait jusqu'à la fin
  - si `pos` manque, on extrait depuis le début :

```
//        0123456789*12
string s("Hello, World!");
string s1 = s.substr(0, 5);     // "Hello"
string s2 = s.substr(7, 5);     // "World"
string s3 = s.substr(7);        // "World!"
string s4 = s.substr();         // "Hello, World!"
```

Recherche



- La **recherche** dans une chaîne est une des opérations les plus courantes, pour laquelle on dispose de 6 méthodes :

`find`	Trouve une sous-chaîne
`rfind`	Idem depuis la fin
`find_first_of`	Trouve le premier caractère parmi une liste
`find_last_of`	Idem depuis la fin
`find_first_not_of`	Trouve le premier caractère hors d'une liste
`find_last_not_of`	Idem depuis la fin

- Toutes ces méthodes renvoient la **position** trouvée (de type `size_t`).
- Si la recherche est **infructueuse**, elles renvoient `string::npos` (= -1, donc `numeric_limits<size_t>::max()`).

find (rfind)



- `.find(string str, size_t pos);` (`rfind`) trouve la première (dernière) occurrence de la sous-chaîne `str`, en cherchant depuis la position `pos` :

- `str` peut être remplacé par un `char`, une constante littérale (`const char*`) ou par ses premiers caractères.
- `pos` vaut zéro si elle n'est pas spécifiée.

```
//        0123456789*123456789*123456789*123456789*123456789*1
string s("There are two needles in this haystack with needles.");
string s2("needle");

size_t needle1   = s.find(s2);
size_t needle2   = s.find(s2, needle1 + 1);
size_t haystack  = s.find("haystack");
needle1          = s.find("needles are small", 0, 6);
needle2          = s.find("needles are small", needle1 + 1, 6);
size_t point     = s.find('.');

s.replace(s.find(s2), s2.length(), "preposition");
cout << s << endl;
```

```
There are two prepositions in this haystack with needles.
```